

Difference Between Agent Harnesses & Agent Frameworks

Tony Kipkemboi
@tonykipkemboi · 9. März · 4 Min. Lesezeit

18 69 564 48.696

If you've been hearing ***"agent framework"*** and ***"agent harness"*** thrown around and can't tell the difference, this is for you. The terms sound interchangeable but they're not. I worked at @crewAIInc, which is an agent framework, so I have a sense of where the boundaries are.

Agent frameworks and agent harnesses sit at different points on a spectrum of opinionation. Understanding where they sit matters if you're building agents, because it changes what you're responsible for and what the tool handles for you.

If you put agent development on a line, raw code with no abstractions sits on the far left. You're calling APIs directly, managing state yourself, building every piece from scratch. Total flexibility, total responsibility.

Agent frameworks sit in the middle. They give you structure and abstractions, but you still make a lot of decisions. You pick the memory system, you configure the tools, you define the orchestration logic. The framework has opinions about how things should connect, but it's modular. You can swap components.

Agent harnesses sit on the far right. They're maximally opinionated. Everything is baked

in. You add your API keys, maybe point it at a few tools, and it runs. Memory, context management, the agent loop, safety checks. All of that is decided for you.

A framework gives you abstractions for building agents. You define roles, tasks, tools. You specify how agents coordinate, whether they work sequentially or hierarchically. The framework handles the plumbing. Calling the LLM, routing tool outputs, managing the execution loop. But you're still making architectural decisions.

The framework is opinionated about what the building blocks look like. It has a memory abstraction, a tool interface, a task structure. But those pieces are swappable. If you don't like the default memory implementation, you can plug in your own. If you want to use a different LLM provider, you configure it. The framework gives you a standard interface, but you're still composing the system.

That modularity is the point. Frameworks are built for people who want to build agents, not just use them. You're expected to understand how the pieces fit together, because you're the one deciding which pieces to use.

A harness doesn't give you building blocks. It gives you a complete system.

The best recent example is @openclaw, which went viral a few weeks ago. It's a harness. You download it, add your API keys, and suddenly you have an agent you can chat with on WhatsApp, Telegram, and other platforms. Memory is handled. Context management is handled. The agent loop is handled. Tool calling, permissions, state persistence. All of it is built in.

You're not configuring a memory system. You're not deciding how tools get registered or how the agent recovers from errors. Those decisions were made by whoever built the harness. Your job is to point it at a task and let it run.

That's the tradeoff. You get something that works immediately, but you don't get to change how it works under the hood. The harness has an opinion about everything, and you're accepting that opinion when you use it.

The spectrum matters because it maps to different problems.

If you're prototyping, experimenting, or building something custom, you want a framework. You need the flexibility to swap components, test different approaches, and control the details. The framework gives you structure without locking you in.

If you need something that works now, reliably, for a specific use case, you want a harness. You're trading control for speed. The harness has already solved the hard problems. Context management, durable execution, error recovery. You're just using the

solution.

Frameworks are for builders. Harnesses are for users.

That doesn't mean one is better. It means they're solving different problems. What you're solving determines which one you need. The line isn't always clean, and I'm not sure it should be.

Some frameworks are adding harness-like features. @LangChain is a good example. They released Deep Agents, which they explicitly call an "agent harness" that sits on top of their framework. It comes with built-in planning tools, file system access for context management, subagent spawning, and memory persistence. You're still using LangChain under the hood, but Deep Agents gives you batteries-included defaults so you don't have to wire everything together yourself.

LangChain actually distinguishes between three layers in their own stack**. LangChain** (the original library) is the **framework**. **LangGraph** is what they call the **"agent runtime,"** which handles **execution, state management, and durability**. **Deep Agents** is the harness that sits on top of both. That's one company spanning the entire spectrum. Framework for composing agents, runtime for executing them reliably, harness for using them out of the box.

That's a framework company moving right on the spectrum. Deep Agents is still modular. You can swap backends, configure tools, adjust prompts. But it gives you a working system without requiring you to assemble every piece.

On the flip side, harnesses aren't as locked down as they might sound. Take OpenClaw. It's maximally opinionated out of the box, but if you download the source code, you can swap implementations. You can change how memory works, adjust the agent loop, modify tool handling. It's just that most people won't, because the default already works.

The distinction is about what's already decided when you start. A harness ships with decisions baked in. A framework ships with options exposed. If you're using a harness, you're accepting most of those decisions and configuring around the edges. If you're using a framework, you're making those decisions yourself and assembling the system.

What you're solving determines which one you need. Sometimes (really most of the time) you need to bypass the agent frameworks entirely and build a simple ReAct agent using the model endpoints directly.

How much you want already built determines which one you pick.



Tony Kipkemboi
@tonykipkemboi

ai agents & oss // ai builders @nebiusai & @nebiustf // army vet // msc @brownuniversity